

Probe-Space Preconditioning for Fast and Stable Zero-Order Training

Anonymous Authors¹

Abstract

Backpropagation (BP) dominates deep learning but imposes a massive memory tax. For example, training OPT-30B with Adam requires ≈ 600 GB of GPU memory (assuming batch size 8 and sequence length 2048). Alternatively, zero-order optimization (ZOO) trains in inference-mode (requiring only ≈ 60 GB for the same model): no stored activations, no gradients, and no optimizer states. However, ZOO convergence has lagged behind BP. In this work, we evaluate two methods to close this gap. First, we show that reallocating training compute budget from many steps to large effective batch sizes with many perturbations (or probes) but fewer steps, allows 1SPSA (Spall, 1992) to outperform zero order methods like MeZO (Malladi et al., 2023) with less training compute. Next, we introduce **1.5-SPSA**, adding a single “clean” forward-pass per step to 1SPSA to calculate a cheap diagonal preconditioner in probe-space, which improves convergence rate and convergence by down-weighting high curvature directions. Benchmarking on 6 post-training datasets on both Qwen3 and OPT model families, we show that 1.5-SPSA achieves State-of-the-Art results over previous ZOO solvers with far less training compute. For example, we train OPT-13B (for direct comparison to MeZO) and find 1.5-SPSA achieves +3.1% accuracy on SST-2 over both MeZO and BP in only 70 optimization steps vs. MeZO’s 100,000 steps. Finally, we combine an 8-bit-packing random generator, triton fused unpack/apply kernels, and distributed parallelism to achieve fast and stable training of models as large as OPT-30B in-place on commodity GPUs (e.g. A100).

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Current deep learning (DL) solvers typically rely on Backpropagation (BP) combined with adaptive optimizers like Adam (Kingma & Ba, 2015). While effective, this paradigm incurs a massive memory tax. For example, training OPT-30B with Adam requires ≈ 600 GB of GPU memory (assuming batch size 8 and sequence length 2048). This prohibits training on a single accelerator for large models, forcing the use of large, expensive accelerator clusters and complex distributed model sharding.

Additionally, neural network loss landscapes typically exhibit highly ill-conditioned Hessians with massive eigenvalue spreads (Ghorbani et al., 2019; Sagun et al., 2017) requiring adaptive optimizers like Adam for stable training. This is exacerbated in deep settings like Reinforcement Learning or Large Language Model (LLM) post-training (Dauphin et al., 2014).

This provides the following desiderata for a new solver:

1. **Inference-Mode Memory Use:** require no saved activations, gradients, or moving averages
2. **Robust to Ill-Conditioned Loss:** capable of converging quickly even in ill-conditioned loss landscapes as is common in training neural networks
3. **Efficient Training Compute:** train models with efficiency, measured as the amount of performance gain per floating-point operation

Derivative-Free Optimization (DFO), or Zero-Order Optimization (ZOO), is a promising area as training is done in inference-mode: no stored activations, no gradients, and no optimizer states. 1SPSA (Simultaneous Perturbation Stochastic Approximation) (Spall, 1992), one of the most famous ZOO methods, performs central difference approximation along a random perturbation (or probe) to estimate gradients versus calculating derivatives and trains in inference-mode, minimizing GPU memory use. Training OPT-30B with 1SPSA, for example, requires only ≈ 60 GB, effectively reducing the memory footprint by $\approx 10\times$ compared to Adam. However, 1SPSA requires many forward-passes, which comes with more compute per step. Additionally, 1SPSA introduces large perturbation noise on top of Adam’s batch noise, both of which must be mitigated to ensure stable

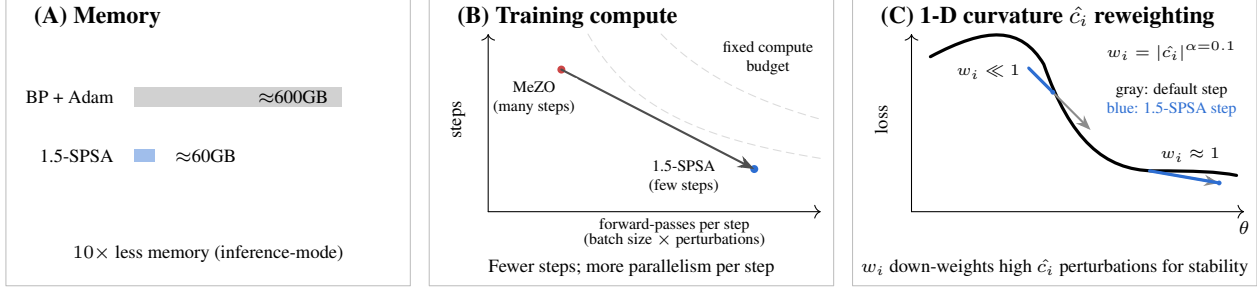


Figure 1. Overview of 1.5-SPSA. (A) Inference-mode training avoids optimizer state and stored activations, reducing memory relative to BP+Adam. (B) forward-passes are reallocated from many optimization steps to large (parallelizable) batch size $\times$ perturbations per step for $44\times$ less training compute to achieve SOTA test accuracy via a ZOO method (on SST-2). (C) Directional curvature estimates $\hat{c}_i$ set weights $w_i = |\hat{c}_i|^{\alpha=0.1}$; high-curvature directions cause instability so steps are down-weighted, while flat regions yield $w_i \approx 1$, which improves stability, convergence rate, and test accuracy with inference-mode memory use.

convergence which we analyze in the post-training setting in Section A.1. Many recent methods have attempted to solve this, most notably MeZO (Malladi et al., 2023) which adapts 1SPSA for LLM post-training in-place, requiring 2 forward-passes per step for 100,000 steps. In this paper, we discover two novel and independent ways to increase convergence beyond MeZO and in some cases BP, with more parallelization and less total training compute.

For our first contribution, we study where 1SPSA should spend its forward-passes: on more optimization steps, or on more computation per step. Under a fixed forward-pass budget $F_{1\text{SPSA}} = s \times a \times 2 \times n_{\text{pert}}$, we sweep batch size (via gradient accumulation at micro-batch = 16) and n_{pert} while holding $F_{1\text{SPSA}} = 500,000$ on OPT-13B SST-2. Figure 2 shows that reallocating budget toward larger effective batch size and more perturbations per step yields substantially higher accuracy in only tens of steps. For example, at effective batch size = 128 and $n_{\text{pert}} = 160$, 1SPSA reaches 94.2% in 80 steps using 205k forward-passes, versus MeZO’s 91.4% which uses similar forward-passes and +2.8 points. However, this regime is parallelizable across perturbations and accumulation steps so 1SPSA is capable of far faster wall-clock training time. Despite strong early progress, 1SPSA can become unstable late in training as curvature varies dramatically across random directions. Adam-style diagonal preconditioning would typically mitigate this, but storing $O(d)$ moments is incompatible with our inference-mode desiderata and estimating in only tens of steps is unlikely to assist convergence.

For our second contribution, we introduce **1.5-SPSA**, adding a single unperturbed loss evaluation per step to estimate a cheap preconditioner in probe-space, that down-weights high-curvature probes described in Section 4. This stabilizes training and permits larger step sizes. On OPT-13B SST-2, 1.5-SPSA reaches 94.5% in 70 steps using 179k forward-passes, outperforming MeZO by +3.1 points with less compute and exceeding BP (+2.5 points), as shown in

Figure 2. To ensure this is not just a property of OPT, we also test on Qwen3-1.7B/8B. Table 3 and Table 4 shows similar performance improvement over 1SPSA and BP as 1.5-SPSA expands the stable learning-rate range and improves convergence rates and convergence.

For our third contribution, we investigate the root cause of these gains in controlled stress tests, where we discover 1.5-SPSA’s improvement over 1SPSA is proportional to the loss landscapes’s hessian condition number κ . Curvature reweighting in high κ losses yields up to $6\times$ fewer steps-to-zero loss and little effect in very low κ as described in Section 5.

Finally, for our fourth contribution, we keep training in-place and fast with the following innovations. We combine an 8-bit-packing random generator, triton fused unpack/apply kernels, and distributed parallelism to minimize wall-clock per step, enabling inference-mode training on models as large as OPT-30B on commodity accelerators (e.g. A100) described in Algorithm 1 and Section A.8.

2. Background: Stochastic Approximation and Finite Differences

Stochastic Approximation (SA) finds roots of noisy functions. Robbins-Monro (1951) introduced the field for noisy gradients, proposing the iterative update:

$$\theta_{k+1} = \theta_k - \lambda \hat{g}(\theta_k) \quad (1)$$

where $\hat{g}$ is a noisy gradient estimate for model weights $\theta \in \mathbb{R}^d$ for loss function $L(\theta)$ and a step size λ . Kiefer-Wolfowitz (1952) extended this to *noisy function values* using component-wise finite differences. However, the Kiefer-Wolfowitz estimator requires $2d$ forward-passes for a d -dimensional gradient, which is prohibitively expensive for deep learning in large models (e.g. $d \sim 10^9$).

To overcome the dimensionality curse, Spall (1992) introduced **Simultaneous Perturbation Stochastic Approximation**.

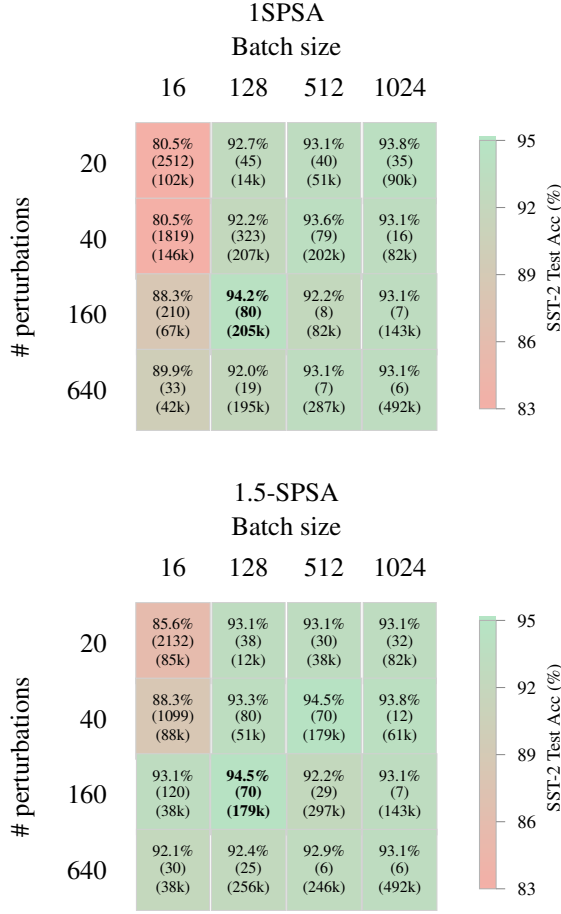


Figure 2. Comparison of 1SPSA (top) and 1.5-SPSA (bottom) for OPT-13B sweeping over n_{pert} and batch size per optimization step. Each cell reports: test accuracy on SST-2 (top), optimization steps to convergence (middle), forward-passes to convergence (bottom). Convergence is defined to be when validation loss has plateaued. All configurations are compared at equal total forward-pass budgets (500,000 forward-passes), although convergence may happen before the final optimization step. We find more batch size and perturbations per step seems to increase convergence rate, but only improves final convergence value up to a point of diminishing returns. We also find that 1.5-SPSA outperforms 1SPSA in both final convergence and convergence rate.

mation (SPSA). SPSA approximates the gradient using a random perturbation vector $z \in \mathbb{R}^d$ and only two function evaluations (forward-passes), independent of dimension d to perform central difference approximation.

1SPSA. The standard gradient estimator for 1SPSA is:

$$\hat{g}(\theta) = \frac{1}{2n_{pert}} \sum_{i=1}^{n_{pert}} \frac{L(\theta + \epsilon z_i) - L(\theta - \epsilon z_i)}{2\epsilon} z_i^{-1} \quad (2)$$

for $\epsilon > 0$ and n_{pert} perturbations, where z_i^{-1} is the coordinate-wise inverse. This is typically sampled from the **Rademacher distribution** ($z_i \in \{-1, +1\}$), so $z^{-1} = z$, simplifying implementation, minimizing memory, and

speeding up convergence to the true gradient (Spall, 1992).

2SPSA. Spall (1997) later introduced 2SPSA to estimate a Hessian preconditioner (Spall, 1997) in parameter space. However, estimating a $d \times d$ Hessian is impossible for modern LLMs. Standard optimizers like Adam estimate a diagonal preconditioner, which is more tractable, but still requires $O(d)$ memory (Kingma & Ba, 2015). We seek a solver that does not require additional $O(d)$ memory use.

2.1. Modern ZOO for neural networks

The main goal of modern ZOO methods adapted to train large neural networks is to reduce both (i) minibatch noise and (ii) perturbation/estimator noise, while keeping memory overhead reasonable. The way they do so differs. We breakdown the main techniques used in modern ZOO methods below.

Structured perturbations for LLMs. A practical obstacle in large models is that a single global perturbation can mix parameters with very different scales across layers. A MeZO variant called LeZO (Wang et al., 2024) addresses this in LLM post-training by applying perturbations in a structured way (layerwise), improving stability at a fixed query budget. Related work explores other structured sparsity strategies for scaling ZO updates in large models like DeepZero (Chen et al., 2024).

Adaptive moment methods. Inspired by first-order optimizer structure, these approaches maintain momentum and/or adaptive per-coordinate learning rates based on gradient estimates. For example, ZO-AdaMM (Chen et al., 2019) provides convergence guarantees with a dimension-dependent slowdown typical of ZOO estimators. These methods can improve practical convergence, but they generally re-introduce optimizer-state memory tax that our in-place setting aims to avoid.

Learned subspaces. To improve sample efficiency, several methods reduce the perturbation search space by restricting search to a much smaller subspace, or by adapting the sampling distribution of perturbation directions over time. ASEBO learns an “active” subspace for evolution-strategy style gradient estimates (Choromanski et al., 2019), while RSVP proposes a variance-reduced ZO scheme (Gautam et al., 2024). In a complementary theory line, ZOO variance-reduction methods build on SVRG/SPIDER-style ideas and provide improved query complexity for finding approximate stationary points in nonconvex problems.

Evolution strategies (ES). ES methods estimate gradients of a smoothed objective from a population of perturbed parameters and are attractive for highly parallel systems. Classic families include NES (Wierstra et al.,

2008) and CEM (Rubinstein & Kroese, 2004); modern large-scale demonstrations show strong distributed scaling properties (Salimans et al., 2017).

Our method attempts to maintain the simplicity of 1SPSA exploring how far we can go with very slight tweaks to the original algorithm.

3. Motivation

3.1. Training Compute Budget Analysis for 1SPSA

Our first investigation into 1SPSA is to understand the impact of the choice of batch size and number of perturbations per optimization step to determine what is the most efficient use of training compute. We measure training compute as the total number of forward-passes which scales with accumulation steps a , number of perturbations n_{pert} , and optimization steps s . The total number of forward-passes then is calculated as $F_{1SPSA} = s \times a \times 2 \times n_{pert}$. We use matched forward-pass budgets when comparing batch size and perturbation allocations to fairly compare configurations. Many ZOO baselines allocate budget toward large s with small a and small n_{pert} .

For our first observation, we find that budget allocations with small s and large (a, n_{pert}) can reach much stronger accuracy in only *tens of optimization steps*. While it is intuitive that higher batch size and more perturbations would result in more stable training, it is unexpected that it would outperform final convergence for both MeZO and BP in such few optimization steps.

Figure 2 shows sweeps over batch and perturbations for SST-2 on OPT-13B. In 80 iters, 1SPSA is able to beat MeZO (94.5% vs. 91.4% SST-2 test accuracy) at batch size 128 and 160 perturbations per step, which at micro batch size 16 is 205k forward-passes vs. 200k for MeZO. We attribute this increase in accuracy to the large learning rate λ made possible with stable loss landscape measurements. Note, the optimal λ found per run is orders of magnitude higher than the typical λ used in post-training for LLMs. We find $\lambda = 5e^{-4}$ to be stable at batch size 128 and 160 perturbations, while MeZO and BP train with $\lambda = 10^{-6}$, allowing our step size to be $500\times$ larger. We observe this while training Qwen3 as well as shown in Table 4.

For our second observation, we find that tying $\lambda = \epsilon$ results in the most stable training as reported in Table 6. This is intuitive since ϵ measures the loss landscape $L(\theta)$ at a specific radius from θ_k , and λ determines how far to step $\theta_{k+1} = \theta_k - \lambda \hat{g}(\theta_k, \epsilon)$. If we set $\lambda > \epsilon$, we step farther than we have measured, and would potentially be unstable. If we set $\lambda < \epsilon$, we step shorter than we have measured, and would slow convergence in stable loss landscapes or would be unstable in ill-conditioned loss landscapes. Additionally,

this is more numerically-stable as the terms cancel in the update rule.

For our third observations, we observe that more batch size and perturbations per step seems to continually increase convergence rate, but only improves final convergence value up to a point of diminishing returns. Surprisingly, the best test accuracy is not at the largest batch size (1024) and n_{pert} (640) tested, but at batch size (128) and n_{pert} (160). We attribute this to the lack of momentum to get out of local minima. As we have no momentum, we need something to shake out of local minima. Leaving some variance may be the key to allow this to happen.

For our fourth observation, we find that training is unstable at the end of training. Training loss diverges at the end of training and does not recover. While we use a plateau-based learning rate schedule that cuts λ and ϵ both by half after a lack of progress, this does not remedy the issue and admittedly has little to no effect on final performance. To understand the loss landscape we are optimizing, we plot an eight thousand perturbation histogram of our 3-point curvature estimate in Figure 6. We observe that curvature can swing from -40^8 to 30^8 and everywhere in between in the same optimization step. If we step in each direction with the same λ , we will be stepping too little in low curvature dimensions, and too far in high curvature dimensions. With this in mind, we seek a way to down-weight these high curvature dimensions to keep training stable.

3.2. Incorporating Curvature into 1SPSA

In convex optimization, preconditioning with the inverse Hessian H^{-1} (Newton’s method) corrects for ill-conditioning. In deep learning, a full H is often unavailable. First-order methods like Adam approximate diagonal curvature in parameter space. Standard 2SPSA attempts to estimate global Hessian-vector products which is intractable and violates our memory constraint in our desiderata. While we cannot afford such methods that use $O(d)$ additionally memory for the solver, especially in such few optimization steps, perhaps there exists a curvature scheme that can be effective. Since we update along random directions vectors z_i , we can take insight from the Johnson-Lindenstrauss (JL) lemma (Johnson & Lindenstrauss, 1984), which suggests that geometry in high-dimensional space is preserved in random low-dimensional projections with some probability. In this way, we are randomly projecting our optimization problem onto a random subspace spanned by our perturbations. Since we can not practically precondition in the high-dimensional space, perhaps we can precondition the low-dimensional subspace and still improve convergence. We derive in section Section A.5 an extension of JL showing a key result: if JL preserves geometry on a random perturbation set, then the same perturbation set also preserves

the curvature terms (up to controlled distortions), justifying curvature reweighting as a stable perturbation-space preconditioner.

With this in mind, we propose a simpler, cheaper curvature scheme: estimate the scalar curvature only along the perturbation directions z_i . We can achieve this with just one additional forward-pass per optimization step (shared across all perturbations), the clean forward-pass ($L(\theta)$), which sits in the middle of all of our central difference approximations. This evaluation is actually required already to track true training loss so this may be viewed as no additional compute. With this center point, we estimate a 3-point scalar curvature term $\hat{c}_i$ per perturbation direction z_i from $L(\theta - \epsilon z_i), L(\theta), L(\theta + \epsilon z_i)$.

However, as neural loss landscapes are non-quadratic and heavy-tailed (Ghorbani et al., 2019), a direct inverse-curvature step would be unstable if curvature is near zero (exploding step) or extremely large (vanishing step). By clipping the curvature estimate to a safe range, we maintain stability while exploiting local geometry. We desire a sub-linear response to extreme curvature: if curvature doubles, we do not necessarily want the step size to halve, as that might be too aggressive. This motivates a α -saturated weighting scheme described below.

4. Method: 1.5-SPSA

Building on this motivation, we present 1.5-SPSA. We combine the compute allocation budget found in our 1SPSA analysis with a cheap, robust curvature-informed preconditioner in perturbation space to improve convergence rate in ill-conditioned loss landscapes as per our desiderata. Critically, we maintain 1SPSA’s inference-mode memory use as well.

4.1. Directional Curvature and α -Saturation

To implement this, we modify the 1SPSA estimator to include one clean loss evaluation $L(\theta)$ per step (shared across all perturbations). For each perturbation triplet using the standard central difference stencil (Spall, 1992) $\{L(\theta + \epsilon z_i), L(\theta), L(\theta - \epsilon z_i)\}$, we can simultaneously estimate the gradient signal *and* the scalar directional curvature:

$$\hat{c}_i = \frac{L(\theta + \epsilon z_i) - 2L(\theta) + L(\theta - \epsilon z_i)}{\epsilon^2} \approx z_i^\top \nabla^2 L(\theta) z_i \quad (3)$$

However, the Hessian spectrum in neural networks is known to be heavy-tailed (Ghorbani et al., 2019; Sagun et al., 2017). Random perturbations occasionally align with extremely sharp directions (high $\hat{c}_i$) or extremely flat ones ($\hat{c}_i \approx 0$). A naive Newton step $\propto 1/\hat{c}_i$ would be unstable (Dauphin et al., 2014). To robustly handle this, we winsorize the curvature using an α -saturated weighting scheme. We

define a robust weight w_i :

$$w_i = \frac{1}{\max(\lambda_{\text{reg}}, |\hat{c}_i|^\alpha)} \quad (4)$$

where λ_{reg} is a scalar regularization term (we use $\lambda_{\text{reg}} = 1$), and $\alpha \in [0, 1]$ controls the aggressiveness of the preconditioning. $\alpha = 1$ corresponds to a full diagonal Newton step in perturbation space, while $\alpha = 0$ recovers standard 1SPSA. and we test $\alpha \in [10^{-5}, 10^{-3}, 10^{-1}, 0.5, 1]$ and find $\alpha = 0.1$ to be consistently optimal across architectures (OPT and Qwen), tasks and model sizes as show in Section A.6.

4.2. 1.5-SPSA Update Rule

Substituting these definitions back into 1SPSA, we arrive at the simple 1.5-SPSA update rule:

$$\Delta\theta = \frac{1}{2n} \sum_{i=1}^n \left(\frac{L(\theta + \epsilon z_i) - L(\theta - \epsilon z_i)}{\max(\lambda_{\text{reg}}, |\hat{c}_i|^\alpha)} \right) z_i \quad (5)$$

As discussed in Section 3, we tie $\lambda = \epsilon$ so those terms cancel.

The 1.5-SPSA update can also be written compactly as

$$\Delta\theta = -\eta Z W \Delta\ell, \quad (6)$$

where

$$\eta := \frac{1}{2n_{\text{pert}}},$$

$$Z := [z_1 \ z_2 \ \cdots \ z_{n_{\text{pert}}}],$$

$$W := \text{diag}(w_1, \dots, w_{n_{\text{pert}}}),$$

$$\Delta\ell := (\Delta\ell_1, \dots, \Delta\ell_{n_{\text{pert}}})^\top, \quad \Delta\ell_i := L(\theta + \epsilon z_i) - L(\theta - \epsilon z_i).$$

4.3. System Design to Maintain our Desiderata

To compute this update, we first observe that all of the loss values $L(\theta), L(\theta + \epsilon z_i), L(\theta - \epsilon z_i)$ can be calculated in parallel allowing us to achieve wall-clock time per update theoretically faster than BP (as we have no backward pass). The wall-clock time per update is then limited only by the number of accelerators available in the cluster and the speed of the forward-pass. However, z_i must be expressed three times so we have a compute vs. memory tradeoff to consider. Instantiating an $O(d)$ sized random perturbation from a seed can take considerable wall-clock time which can be reduced by caching z_i . However this would violate our inference-only memory use. We must re-instantiate z_i three times as fast as possible. So we combine three innovations to maintain our desiderata:

- **Bit-Packed perturbations:** Since z_i are Rademacher, we instantiate them as 1 bit per parameter and only one

layer block at a time (block-size B configurable). For a 13B model with $B = 1$, a perturbation is ≈ 1.6 GB (unpacked bf16) but only ≈ 1.6 GB / 16 = 100 MB packed.

- **Fused Kernels:** We implement custom fused CUDA/Triton kernels that unpack the perturbation bit, scale it by ϵ or the computed weight w_i , and add it to the weights in-place. This combined with bit-packing provides a $2.76\times$ speedup over pytorch implementation as described in Section A.8.
- **Seed-Based Distribution:** We use distributed training to parallelize the forward passes as outlined in Algorithm 1. We start by broadcasting seed scalars for each perturbation to the ranks. Each GPU generates its perturbation on the fly, calculates the forward-pass, and communicates back only a scalar loss minimizing communication overhead to only scalars. Rank 0 must then regenerate the perturbation, scale it appropriately and update the model. Once the update is completed, we need to communicate the updated model θ to all GPUs.

This results in training that uses nearly the same memory as inference and wall-clock per step that can be scaled up or down depending on the availability of compute. We outline our full implementation in Algorithm 1. In practice, we train on a single 8xA100 node and perturbation forward-passes are fully parallelized across 8 ranks, reducing the sequential forward-pass computations by $8\times$ vs. full parallelization. Rank 0 sends seeds to all other ranks, and each rank applies its assigned perturbations locally, computes losses, and communicates only scalar loss values back to rank 0 to update θ . After the update, the updated θ must be communicated to all ranks but this happens only tens of times with large batch size and n_{pert} . Additionally, its important to note that total cluster memory use actually scales by number of ranks (r) and micro batch size (m) and model size $|\theta|$ providing: $O(rm|\theta|)$ memory use, as is typical in Distributed Data Parallelism (DDP).

5. Experiments

This section reports results for all experiments conducted.

5.1. OPT-13B/30B Post-Training MeZO comparison

First, we observe 1.5-SPSA for post-training LLMs on OPT-13B for direct comparison to MeZO. We also scale this up to OPT-30B to see how we compare to MeZO at larger model sizes. We emphasize that our BP baselines are drawn from standard post-training practice and are not exhaustively re-optimized for the extreme large-batch, few-step regime explored in this section. Our goal is not to claim universal superiority over backpropagation, but rather to demonstrate that 1.5-SPSA outperforms previous zero-order methods,

Algorithm 1 1.5-SPSA.STEP($\theta, \epsilon, a, n, \alpha, \lambda_{reg}$)

```

1: Input:  $\theta \in \mathbb{R}^d, \epsilon > 0$ , accumulation  $a, n_{pert}, \alpha, \lambda_{reg}$ 
2: Output: updated parameters  $\theta$ 
3:  $r \leftarrow \text{DIST.GET\_RANK}()$ 
4:  $\text{DIST.BROADCAST}(\theta, \text{src} = 0)$ 
5: if  $r = 0$  then
6:   Sample seeds  $S = \{s_i\}_{i=1}^{n_{pert}}$  and scatter across ranks
7: end if
8:  $\text{DIST.SCATTER}(S, \text{src} = 0)$ 
9: Compute center loss once:  $L_0 \leftarrow \sum_{t=1}^a \mathcal{L}(\theta, x_t)$ 
10: for  $i = 1$  to  $n$  do
11:   if  $r = i$  then
12:      $z_i \leftarrow \text{PACKEDPERTURBATION}(s_i)$ 
13:      $\theta \leftarrow \text{APPLYPACKEDPERTURBATION}(\theta, +\epsilon, z_i)$ 
14:      $L_+ \leftarrow \sum_{t=1}^a \mathcal{L}(\theta, x_t)$ 
15:      $\theta \leftarrow \text{APPLYPACKEDPERTURBATION}(\theta, -2\epsilon, z_i)$ 
16:      $L_- \leftarrow \sum_{t=1}^a \mathcal{L}(\theta, x_t)$ 
17:      $\theta \leftarrow \text{APPLYPACKEDPERTURBATION}(\theta, +\epsilon, z_i)$ 
18:      $\text{DIST.GATHER}([L_+, L_-], \text{dst} = 0)$ 
19:   end if
20: end for
21: if  $r = 0$  then
22:   for  $i = 1$  to  $n$  do
23:      $z_i \leftarrow \text{PACKEDPERTURBATION}(s_i)$ 
24:      $\hat{c}_i \leftarrow (L_{+,i} - 2L_0 + L_{-,i})/\epsilon^2$ 
25:      $g_i \leftarrow (L_{+,i} - L_{-,i})$ 
26:      $w_i \leftarrow 1/\max(\lambda_{reg}, |\hat{c}_i|^\alpha)$ 
27:      $\gamma_i \leftarrow \frac{1}{2n} g_i w_i$ 
28:      $\theta \leftarrow \text{APPLYPACKEDPERTURBATION}(\theta, -\gamma_i, z_i)$ 
29:   end for
30: end if
    
```

and can achieve competitive or superior performance to BP in specific compute allocation as is done in the MeZO configuration.

Method	SST-2	RTE	BoolQ	WSC	WiC
Zero-shot	58.8	59.6	59.0	38.5	55.0
ICL	87.0	62.1	66.9	39.4	50.5
LP	93.4	68.6	59.3	63.5	60.2
MeZO (Malladi et al., 2023)	91.4	66.1	67.6	63.5	61.1
MeZO (LoRA)	89.6	67.9	73.8	64.4	59.7
MeZO (prefix)	90.7	70.8	73.1	60.6	59.9
BP+Adam	92.0	70.8	77.1	63.5	70.1
1SPSA	94.2	63.3	76.5	65.4	61.8
1.5-SPSA	94.5	77.7	76.5	71.2	61.9

Table 1. OPT-13B post-training accuracy results on common GLUE/SuperGLUE tasks. Rows above the midrule are baselines. All 1SPSA and 1.5-SPSA were trained with less than 300 optimization steps. 1.5-SPSA has superior performance.

Method	SST-2	RTE	BoolQ	WSC	WiC
MeZO / Prefix	90.6	72.6	73.5	63.5	59.1
1SPSA	94.0	69.0	73.0	67.5	59.9
1.5-SPSA	94.5	77.0	74.0	67.5	59.3

Table 2. OPT-30B post-training accuracy results on common GLUE/SuperGLUE tasks. All 1SPSA and 1.5-SPSA were trained with less than 300 optimization steps. 1.5-SPSA proves superior performance.

5.2. Qwen3 Post-Training

To ensure these results are not just a property of OPT, we train on a more modern architecture. We train Qwen3-8B on the same benchmarks as the OPT trials. Also, we train Qwen3-1.7B (Yang et al., 2025) on Stable ToolBench (Guo et al., 2024) to see how it performs on a different task vs. those tested in MeZO. 1.5-SPSA outperforms 1SPSA by 0.4%, and BP with Adam by 2%. Additionally, we perform a sweep of learning rates λ s to demonstrate the stability of 1.5-SPSA over 1SPSA and MeZO at much higher learning rates which allows for larger jumps in parameter space.

Method	SST-2	RTE	BoolQ	WSC	WiC
1SPSA	94.5	88.5	85.7	75.0	64.6
1.5-SPSA	94.7	88.0	86.1	80.8	71.2

Table 3. Qwen3-8B post-training accuracy results on common GLUE/SuperGLUE tasks. All 1SPSA and 1.5-SPSA were trained with less than 300 optimization steps. 1.5-SPSA proves superior performance.

Method	lr=1e-3	lr=5e-4	lr=1e-4	lr=5e-5	lr=1e-5	lr=5e-6
BP+Adam	72.0 (70)	70.2 (80)	77.0 (285)	77.0 (284)	70.2 (279)	65.3 (275)
1SPSA	<i>diverge</i>	<i>diverge</i>	78.6 (38)	78.5 (38)	77.2 (44)	77.2 (45)
1.5-SPSA	77.0 (24)	79.0 (24)	78.0 (37)	77.6 (37)	77.1 (39)	77.0 (44)

Table 4. Qwen3-1.7B post-training accuracy results on Stable ToolBench to teach tool-use. Format: Test Acc (on top), optimization steps to converge (on bottom). All 1SPSA and 1.5-SPSA were trained with less than 300 optimization steps. We show 1.5-SPSA outperforms 1SPSA and BP with Adam on Qwen3. Additionally, we show that 1.5-SPSA permits larger learning rates which improves convergence rate.

5.3. Convex Optimization: Toy Stiff Paraboloid

We hypothesize that the benefit of curvature preconditioning scales with the condition number of the local Hessian. To test this, we construct a synthetic stiff paraboloid objective $J(x) = x^T R^T \text{diag}(k, 1, \dots, 1) R x$, where R is a random orthonormal matrix and k controls the condition number $\kappa = \frac{k}{1}$. We vary $k \in \{1, 5, 10, 50, 100, 500, 1000\}$

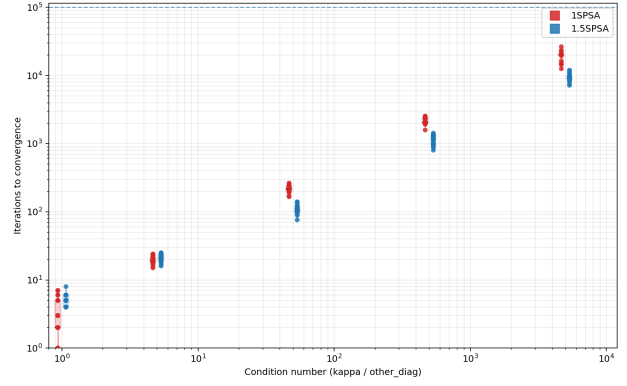


Figure 3. Comparing convergence rates on a stiff paraboloid $x^T R^T \text{diag}(k, 1, \dots, 1) R x$ at varying condition numbers ($\kappa = k/1$). 10 seeds per $\kappa \times \text{solver}$. Random R and perturbations to compare 1SPSA vs. 1.5-SPSA convergence rates. As κ grows, 1.5-SPSA converges faster (up to $7\times$ on average).

and measure the number of optimization steps required to converge to the global minimum x^* . As illustrated in Figure 3, 1.5-SPSA matches 1SPSA’s convergence rate at low k (well-conditioned) but converges significantly faster as k increases (ill-conditioned), confirming our hypothesis.

5.4. Non-Convex Optimization: DNC overfitting stress tests across scale

Differentiable Neural Computers (DNCs), as introduced by (Graves et al., 2016), are a class of Recurrent Neural Networks (RNNs) that are notoriously difficult to train as they include an external memory that can be read from and written to at each step as well as a hidden memory. We check to see if 1.5-SPSA can outperform 1SPSA and BPTT in convergence rate on a fixed batch until loss reaches a near-zero threshold. We report convergence rate (steps-to-0) across model scales up to 1B parameters, and also show the impact of increasing n_{pert} . As shown in Figure 4, while overfitting a DNC model, the larger we make n_{pert} , and as we increase the model size, the faster convergence rate. We observe that 1.5-SPSA converges much faster than 1SPSA (by as much as $6\times$ less steps) and in many cases faster the BPTT at larger perturbations per step.

6. Discussion and Future Work

Limitations and scope of comparison to backpropagation. A limitation of this study is that we do not exhaustively retune backpropagation (BP) baselines under the same extreme compute-allocation regimes explored for 1SPSA and 1.5-SPSA. While such tuning is possible in principle, it typically requires substantial optimizer state, gradient checkpointing, or backward-pass memory, which conflicts with our desiderata. For example, we may achieve considerable

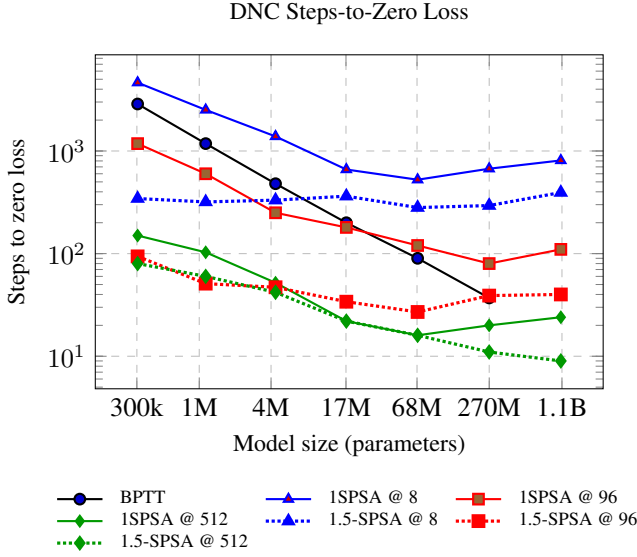


Figure 4. DNC overfitting stress test: steps to reach a near-zero loss for 7 different model sizes up to 1B. 1.5-SPSA reduces steps over 1-SPSA at the same perturbation count. More perturbation count improves convergence rate. Larger models converge faster. BPTT cannot run the 1.1B model in this setup due to GPU memory limits.

reduction in memory using gradient checkpointing at the cost of additional compute. We therefore interpret our results not as a claim that 1.5-SPSA universally outperforms BP on memory, compute or accuracy, but as evidence that when compute is allocated normally, 1.5-SPSA is highly competitive.

Choice of batch size and n_{pert} We perform a detailed analysis of curvature, batch noise and perturbation noise in Section A.1 to help us understand the relationship between our two most important hyperparameters and gradient noise. We can combat batch noise and perturbation noise by increasing batch size and n_{pert} , trading off compute for stability, but how much is enough? We find that stability is related to the median $\hat{g}(\theta)$. We analyze each separately. First, in Figure 8, which shows when batch noise reduces below the median gradient, we see that convergence does well once our signal to noise ratio approaches and surpasses 1. This is the point when we start to see stable convergence in Figure 2. Similarly, in Figure 10, which shows when perturbation noise reduces below the median gradient, we see that convergence does well once our signal to noise ratio approaches and surpasses 1. Rather than brute-force search through these hyperparameters, we can use this technique to find the minimally sufficient batch size and n_{pert} that will surpass $\text{SNR}=1$.

Choice of λ . The learning rate λ plays a critical role in realizing the benefits of 1-SPSA and 1.5-SPSA. If λ is too

small, curvature scaling becomes ineffective and instability can reappear. While we brute force search for λ , a more systematic approach (linear, binary, quadratic fit search) may significantly reduce tuning overhead and improve robustness, as well as offer insight into a better learning rate schedule than plateau decay.

Choice of α . We find that $\alpha = 0.1$ is sufficiently optimal throughout tasks and architectures, which yields mild curvature weighting as shown in Appendix Section A.6. During training, as shown in Figure 5, we observe 3-point curvatures as large as 10^8 . Note, $|\hat{c}|^{\alpha=0.1} \approx 6.3$. So under our weighting scheme, we down-weight this direction $6\times$ more than zero-curvature direction. This mild down-weighting is sufficient to remove instability. Furthermore, these stability gains compound across iterations.

Batch-normalized curvature normalization. Our current curvature scaling operates on per-perturbation curvature estimates without normalization across the batches and/or perturbations. As shown in Figure 6, it may be appropriate to normalize the curvature per batch, or per perturbation, or per step. Exploring batch-normalized or relative curvature estimates is a natural extension and may further stabilize updates without introducing additional hyper-parameters or memory use. We leave this to future work.

7. Conclusion

We presented 1.5-SPSA, a memory-efficient zero-order optimizer that bridges the gap between first-order and zero-order training efficacy. By maintaining constant training compute, we showed that 1-SPSA and 1.5-SPSA perform better with larger effective batch sizes and many perturbations, not many steps. To tame the resulting curvature variance, algorithm 1.5-SPSA adds a single clean forward-pass to precondition updates in perturbation space. This yields State-of-the-Art results on OPT post-training for ZOO with orders of magnitude less compute compared to prior ZOO methods.

References

- Chen, A., Zhang, Y., Jia, J., Diffenderfer, J., Liu, J., Parasyris, K., Zhang, Y., Zhang, Z., Kaikhura, B., and Liu, S. Deepzero: Scaling up zeroth-order optimization for deep model training. In *International Conference on Learning Representations (ICLR)*, 2024.
- Chen, X., Liu, S., Xu, K., Li, X., Lin, X., Hong, M., and Cox, D. Zo-adamm: Zeroth-order adaptive momentum method for black-box optimization. In *Advances in Neural Information Processing Systems*, 2019.
- Choromanski, K., Pacchiano, A., Parker-Holder, J., Tang, Y., and Sindhvani, V. From complexity to simplicity: Adaptive es-active subspaces for blackbox optimization. *arXiv preprint arXiv:1912.01255*, 2019.
- Dauphin, Y. N., Pascanu, R., Gulcehre, C., Cho, K., Ganguli, S., and Bengio, Y. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. In *Advances in Neural Information Processing Systems*, 2014.
- Gautam, T., Park, Y., Zhou, H., Raman, P., and Ha, W. Variance-reduced zeroth-order methods for fine-tuning language models, 2024. URL <https://arxiv.org/abs/2404.08080>.
- Ghorbani, B., Krishnan, S., and Xiao, Y. An investigation into neural net optimization via hessian eigenvalue density. *arXiv preprint arXiv:1901.10159*, 2019.
- Graves, A., Wayne, G., Reynolds, M., Harley, T., Danihelka, I., Grabska-Barwinska, A., Gomez, S., Grefenstette, E., Ramalho, T., Agapiou, J., et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016. doi: 10.1038/nature20101. URL <https://doi.org/10.1038/nature20101>.
- Guo, Z., Cheng, S., Wang, H., Liang, S., Qin, Y., Li, P., Liu, Z., Sun, M., and Liu, Y. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models, 2024. URL <https://arxiv.org/abs/2403.07714>.
- Johnson, W. B. and Lindenstrauss, J. Extensions of Lipschitz mappings into a Hilbert space. In Beals, R., Beck, A., Bellow, A., and Hajian, A. (eds.), *Conference in Modern Analysis and Probability*, volume 26 of *Contemporary Mathematics*, pp. 189–206. American Mathematical Society, Providence, RI, 1984. doi: 10.1090/conm/026/737400.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Malladi, S., Gao, T., Nichani, E., Damian, A., Lee, J. D., Chen, D., and Arora, S. Fine-tuning language models with just forward passes. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2305.17333>. Corrected from early arXiv ID to NeurIPS 2023 version.
- Rubinstein, R. Y. and Kroese, D. P. *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*. Information Science and Statistics. Springer, New York, NY, 2004. doi: 10.1007/978-1-4757-4321-0.
- Sagun, L., Evci, U., Guney, V. U., Dauphin, Y., and Bottou, L. Empirical analysis of the hessian of over-parametrized neural networks. *arXiv preprint arXiv:1706.04454*, 2017.
- Salimans, T. et al. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv*, 2017.
- Spall, J. C. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, 37(3):332–341, 1992. doi: 10.1109/9.119632.
- Spall, J. C. Accelerated second-order stochastic optimization using only function measurements. In *Proceedings of the 36th IEEE Conference on Decision and Control*, volume 2, pp. 1417–1424. IEEE, 1997. doi: 10.1109/CDC.1997.657661.
- Wang, F., Shen, L., Ding, L., Xue, C., Liu, Y., and Ding, C. Simultaneous computation and memory efficient zeroth-order optimizer for fine-tuning large language models. *arXiv preprint arXiv:2410.09823*, 2024.
- Wierstra, D. et al. Natural evolution strategies. *Journal of Machine Learning Research*, 2008.
- Yang, A. et al. Qwen3: The next generation of unified large language models. *arXiv preprint arXiv:2505.09388*, 2025.

A. Appendix

A.1. Empirical Analysis on Qwen3-8B Loss Landscape

In this section, we analyze the Qwen3-8B Loss Landscape subject to SST-2.

A.1.1. 1-D LOSS LANDSCAPE

We first attempt to understand the loss landscape. We take random directions z_i and plot 40 points along the that direction. We do this 100 times and show the line plots in Figure 5. As you can, the lines are very non-convex, with high variance in curvature.

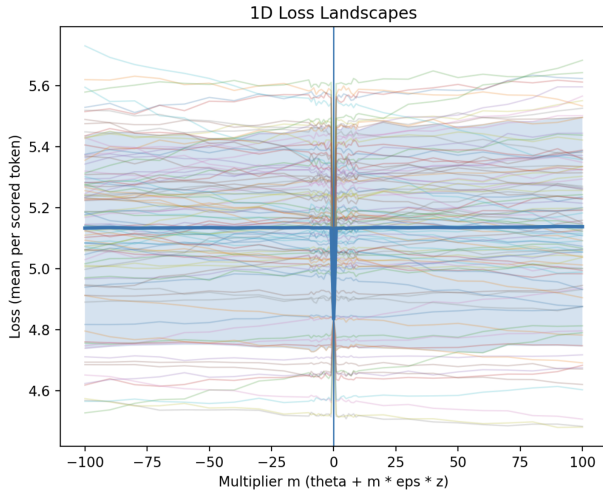


Figure 5. We perturb Qwen3-8B in random directions and plot the 1-D loss landscape at 40-points (SST-2, batch size = 256). The resulting “spaghetti plot” shows an **indefinite and ill-conditioned Hessian**: local curvature ranges from sharply negative to sharply positive (up to 10^8).

A.1.2. 3-POINT CURVATURE HISTOGRAM

Next, we attempt to characterize the distribution of our 3-point curvature. We probe eight thousand random directions and calculate $\hat{c}$ as defined above and plot the histogram in Figure 6. As shown, the values can range by eight orders of magnitude. This demonstrates that our loss function is very ill-conditioned and suggests preconditioning would be useful for faster convergence.

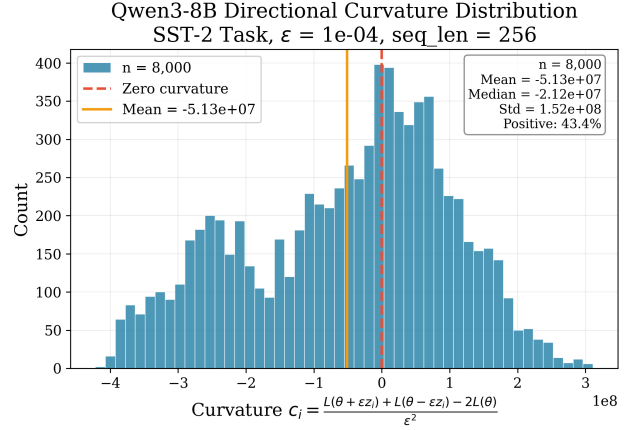


Figure 6. **Histogram of 3-point curvature of Qwen3-8B loss landscape against SST-2.** We see values ranging from -40^8 to 30^8 proving the post-training loss is ill-conditioned. We use batch size = 128, sequence length = 256, and $\epsilon = 10^{-4}$ for eight thousand random perturbation directions.

A.1.3. BATCH NOISE ANALYSIS

Next, we want to understand the impact of batch size on our gradient estimate. Intuitively, we know we can decrease the noise in our gradient estimate with larger batch size, but as a practitioner, we still need to know how big is big enough for us to achieve stable convergence. We plot the distribution of our gradient estimate at different batch sizes in Figure 7. Additionally, in Figure 8, we also plot the median gradient as well for us to get a sense of the signal we are trying to discern from the noise. The variance in our gradient estimate meets our median gradient around batch size 128 to 256 providing our target range for optimization. We also find that batch size less than this amount leads to unstable training.

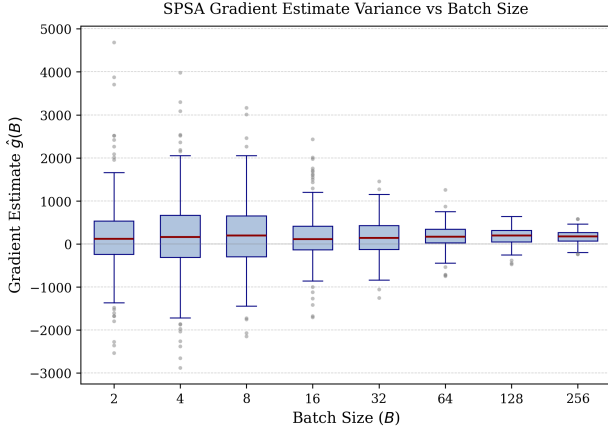


Figure 7. Box plot showing the distribution of gradient estimates for different batch size. We measure the finite difference gradient estimate using a Qwen3-8B model on the SST-2 sentiment classification task. A single Rademacher perturbation vector $z \in \{-1, +1\}^d$ is sampled once (seed = 0) and held fixed across all measurements. For each batch size $B \in \{2, 4, 8, 16, 32, 64, 128, 256\}$, we sample $n = 200$ independent batches B_j from the training set and compute $\hat{g}(B_j)$ using $\epsilon = 10^{-4}$. The same perturbation z and perturbed models $\theta \pm \epsilon z$ are used for every batch; only the data batch varies.

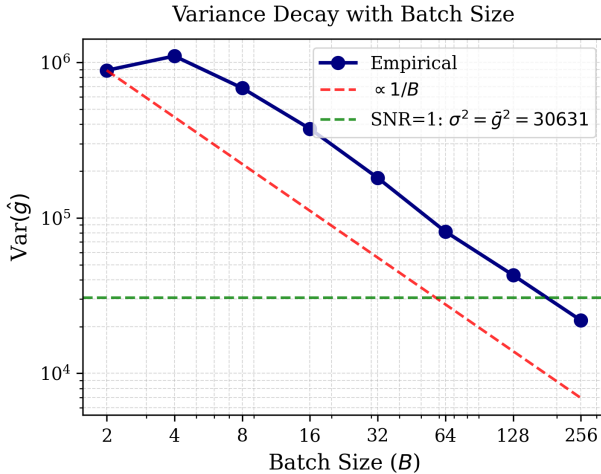


Figure 8. Empirical variance follows the expected $O(1/B)$ decay (red dashed line), consistent with the central limit theorem applied to mini-batch loss averaging. The green dashed line indicates the $\text{SNR} = 1$ threshold where the gradient signal magnitude equals the noise standard deviation.

A.1.4. PERTURBATION NOISE ANALYSIS

Next, we want to understand the impact of perturbation noise on our gradient estimate the same way we did for batch noise. Intuitively, we know we can decrease the noise in our gradient estimate with larger perturbations, but as a practitioner, we still need to know how big is big enough for us to achieve stable convergence. We plot the distribution

of our gradient estimate at different perturbations per step in Figure 9. Additionally, in Figure 10, we also plot the median gradient as well for us to get a sense of the signal we are trying to discern from the noise. The variance in our gradient estimate meets our median gradient around perturbations 256.

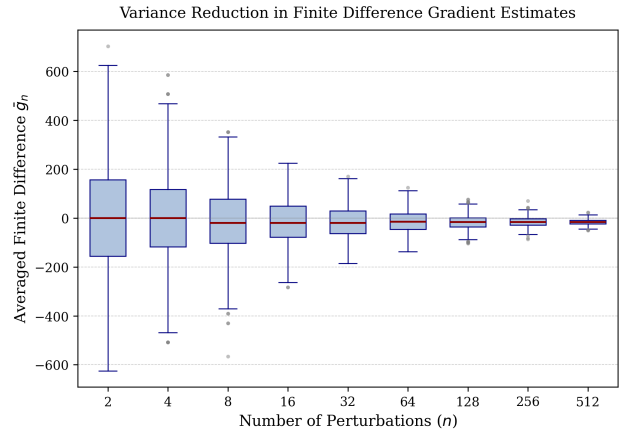


Figure 9. Variance reduction in finite difference gradient estimates as a function of the number of perturbations. We load Qwen3-8B and fix a single batch $\mathcal{B}$ of size 128 from SST-2. We generate 1000 independent Rademacher perturbations $\{z_i\}_{i=1}^{1000}$ and compute our gradient estimate with $\epsilon = 10^{-4}$. The same batch $\mathcal{B}$ is used for all 2000 loss evaluations, isolating perturbation variance from batch variance. For each $n_{\text{pert}} \in \{2, 4, 8, 16, 32, 64, 128, 256, 512\}$, we bootstrap 1000 times by sampling n perturbations without replacement. Variance decreases as $O(1/n_{\text{pert}})$.

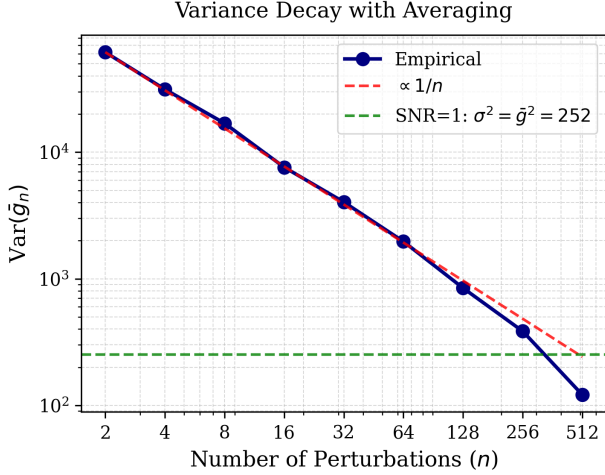


Figure 10. Empirical variance of the averaged finite difference estimator $\bar{g}_n$ as a function of n_{pert} . Same experimental setup as Figure 9. Blue circles show empirical variance from 1000 bootstrap samples at each n_{pert} . Red dashed line shows the theoretical $O(1/n_{pert})$ decay. Green dashed line marks $\text{Var}(\bar{g}_{n_{pert}}) = \bar{g}^2 \approx 252$, the $\text{SNR} = 1$ threshold where standard deviation equals signal magnitude. The estimator crosses into the $\text{SNR} > 1$ regime around $n_{pert} \approx 256$. Deviation from $1/n_{pert}$ at large n_{pert} is due to finite population effects (sampling 512 of 1000 total perturbations without replacement).

A.2. Convex Optimization: Stiff Paraboloid Experiments

To better study the conditions by which 1.5-SPSA outperforms 1SPSA, we construct a toy convex optimization problem. We create a paraboloid that must be rotated randomly as we use the Rademacher distribution and we want to examine which solver outperforms in well-conditioned loss landscapes, where the eigenvalues of the hessian are roughly equal, and which solver outperforms in ill-conditioned loss landscapes, where the eigenvalues of the hessian are orders of magnitude different. We see an example setup in Figure 11 where we train with each solver until convergence. We randomize this experiment, and run one hundred runs each condition number in a sweep of condition numbers to see a clear pattern as shown in Figure 3. As we increase the condition number, 1.5-SPSA shows faster and faster convergence over 1SPSA providing some evidence that the performance increase is tied to the condition number of the hessian.

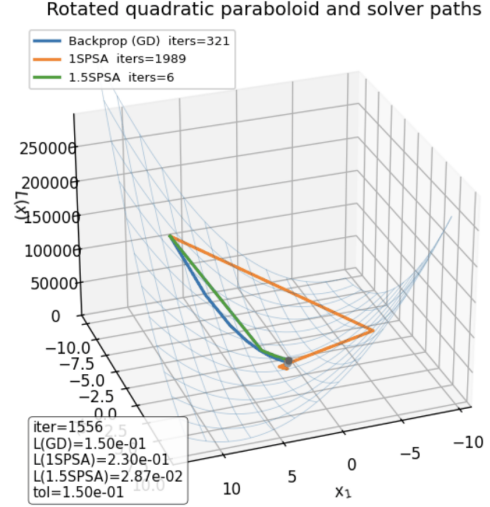


Figure 11. Comparing convergence rates for our “stiff” paraboloid with a $\kappa = 100$. 1.5-SPSA converges in just 6 steps while 1SPSA takes ~ 2000 steps.

A.3. Non-Convex: DNC Overfit Compute Comparison

Differentiable Neural Computers (DNCs) are notoriously difficult to train. To test 1.5-SPSA, we train on a sweep of DNC model sizes up to 1B parameters and for each one, overfit on a batch similar to the stiff paraboloid experiment but in this case the loss is massively non-convex. We train to convergence with 1.5-SPSA, 1SPSA, and BPTT (Backpropagation Through Time). We see that 1.5-SPSA outperforms 1SPSA for all runs. We plot in Figure 12 the total forward-passes to achieve near zero train loss, estimating a backward pass equals roughly two forward passes, therefore one step of BPTT costs three forward-passes. Despite this, BPTT mostly outperforms on this task suggesting BPTT is a more compute-efficient solver, at least on this task, yet 1.5-SPSA is more compute-efficient solver than 1SPSA. Additionally, while mostly, BPTT (Backpropagation through time) outperforms, there are cases where 1.5-SPSA requires less compute to achieve near zero loss (e.g. at small model sizes and 512 perturbations per step).

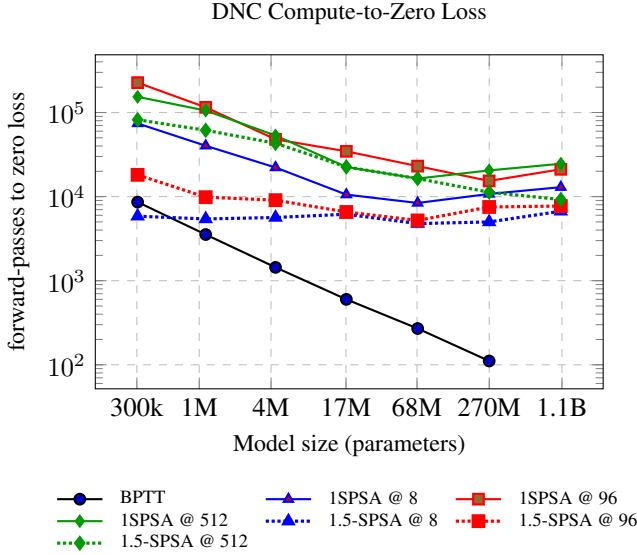


Figure 12. DNC overfitting stress test: compute (measured as forward-passes or equivalent) to reach a near-zero loss threshold vs model size. We convert steps to forward-passes as follows: 1SPSA uses $2n_{\text{pert}}$ forward-passes per step, 1.5-SPSA uses $(2n_{\text{pert}} + 1)$ forward-passes per step, and BPTT is approximated as 3 forward-pass equivalents per step (forward + backward). BPTT cannot run the 1.1B model in this setup due to GPU memory limits.

A.4. Experiment Configurations

Unless stated otherwise, all experiments use the following shared setup. We use the above configuration for: OPT-13B, OPT-30B, Qwen3-8B, and Qwen3-1.7B.

Zero-order methods (1SPSA / 1.5-SPSA). We sweep a tied step size and perturbation size, $\lambda = \epsilon$, over

$$\lambda = \epsilon \in \{10^{-3}, 5 \cdot 10^{-4}, \dots, 10^{-7}\}.$$

We fix the saturation exponent to $\alpha = 0.1$ for all runs as explained in Table 5, and apply a plateau schedule that simultaneously cuts both λ and ϵ when the validation loss fails to improve for 10 consecutive evaluations. We do not use any learning-rate warmup. We sweep the number of perturbations per step $n_{\text{pert}} \in \{40, 60, 100\}$ and the effective batch size $\in \{128, 256\}$ (via gradient accumulation if needed), with sequence length 256 unless noted otherwise. In practice, $n_{\text{pert}} = 60$ and $\lambda = \epsilon = 10^{-4}$ is typically sufficient for stable 1.5-SPSA performance across our settings.

Backpropagation baseline (Adam). For Adam runs, we use $(\beta_1, \beta_2) = (0.9, 0.99)$ with weight decay 10^{-3} .

Hardware. All runs are executed on a cluster of $8 \times \text{A100}$ GPUs.

DNC setting. For DNC experiments, we use the same as above except we use a character-level tokenizer, sequence

length 100, an input embedding size of 128, and add $\lambda = \epsilon = 10^{-2}$ in the sweep for these runs as the weights are xavier-initialized.

A.5. JL preservation of directional curvature

This appendix formalizes the following intuition used in our method: *if a Johnson–Lindenstrauss (JL) embedding preserves geometry (distances / inner products) on a set of perturbation directions, then it also preserves the directional curvature terms that define our probe-space preconditioner, with high probability.* Throughout, fix a point $\theta \in \mathbb{R}^d$ and assume $L : \mathbb{R}^d \rightarrow \mathbb{R}$ is twice differentiable in a neighborhood of θ with Hessian $H := \nabla^2 L(\theta) \in \mathbb{R}^{d \times d}$ (symmetric, possibly indefinite).

Lemma A.1 (JL inner-product preservation (normalized, pairwise form)). *Let $R \in \mathbb{R}^{m \times d}$ be a linear map (as in standard JL transforms: Gaussian, subgaussian, SRHT, CountSketch, etc.). Fix a finite collection of pairs $\{(x_i, y_i)\}_{i=1}^n$ with $x_i \neq 0$ and $y_i \neq 0$, and define unit vectors $u_i := x_i / \|x_i\|$ and $v_i := y_i / \|y_i\|$. Assume the JL norm guarantee holds simultaneously on the finite set*

$$\mathcal{S} := \{u_i, v_i, u_i + v_i, u_i - v_i\}_{i=1}^n,$$

namely for some $\varepsilon_{\text{JL}} \in (0, 1)$,

$$(1 - \varepsilon_{\text{JL}}) \|s\|^2 \leq \|Rs\|^2 \leq (1 + \varepsilon_{\text{JL}}) \|s\|^2 \quad \forall s \in \mathcal{S}. \quad (7)$$

Then for every $i \in \{1, \dots, n\}$,

$$|\langle Rx_i, Ry_i \rangle - \langle x_i, y_i \rangle| \leq \varepsilon_{\text{JL}} \|x_i\| \|y_i\|. \quad (8)$$

Proof. For each i , apply polarization to the unit vectors u_i, v_i :

$$\begin{aligned} & \langle Ru_i, Rv_i \rangle - \langle u_i, v_i \rangle \\ &= \frac{1}{4} \left([\|R(u_i + v_i)\|^2 - \|u_i + v_i\|^2] - [\|R(u_i - v_i)\|^2 - \|u_i - v_i\|^2] \right). \end{aligned}$$

Using (7), we have $|\|R(u_i \pm v_i)\|^2 - \|u_i \pm v_i\|^2| \leq \varepsilon_{\text{JL}} \|u_i \pm v_i\|^2$, hence

$$\begin{aligned} |\langle Ru_i, Rv_i \rangle - \langle u_i, v_i \rangle| &\leq \frac{\varepsilon_{\text{JL}}}{4} (\|u_i + v_i\|^2 + \|u_i - v_i\|^2) \\ &= \frac{\varepsilon_{\text{JL}}}{4} \cdot 4 \\ &= \varepsilon_{\text{JL}}. \end{aligned} \quad (9)$$

Multiplying by $\|x_i\| \|y_i\|$ yields (8). $\square$

Remark (always-valid weaker bound without normalization). If instead you assume JL norm preservation on $\{x_i, y_i, x_i \pm y_i\}$, then polarization yields the bound

$$|\langle Rx_i, Ry_i \rangle - \langle x_i, y_i \rangle| \leq \frac{\varepsilon_{\text{JL}}}{2} (\|x_i\|^2 + \|y_i\|^2),$$

which is weaker than (8) when $\|x_i\|$ and $\|y_i\|$ are very different.

Theorem A.2 (JL preservation of directional curvature on a finite set). Fix $\theta \in \mathbb{R}^d$ and a twice-differentiable L near θ with Hessian $H := \nabla^2 L(\theta) \in \mathbb{R}^{d \times d}$. Fix directions $\{z_i\}_{i=1}^n$ and define $v_i := Hz_i$ and the (true) directional curvature

$$c_i := z_i^\top Hz_i = \langle z_i, v_i \rangle.$$

Let $R \in \mathbb{R}^{m \times d}$ be a (linear) JL transform (e.g., i.i.d. Gaussian entries $R_{jk} \sim \mathcal{N}(0, 1/m)$). Define the JL curvature proxy

$$\tilde{c}_i := \langle Rz_i, Rv_i \rangle = \langle Rz_i, R(Hz_i) \rangle.$$

Let $\varepsilon_{\text{JL}} \in (0, 1)$ and $\delta \in (0, 1)$. If

$$m \gtrsim \varepsilon_{\text{JL}}^{-2} \log \frac{n}{\delta}, \quad (10)$$

then with probability at least $1 - \delta$ over R , simultaneously for all $i \in \{1, \dots, n\}$,

$$|\tilde{c}_i - c_i| \leq \varepsilon_{\text{JL}} \|z_i\| \|Hz_i\|. \quad (11)$$

Moreover, whenever $c_i \neq 0$,

$$\frac{|\tilde{c}_i - c_i|}{|c_i|} \leq \varepsilon_{\text{JL}} \cdot \frac{\|z_i\| \|Hz_i\|}{|\langle z_i, Hz_i \rangle|} = \frac{\varepsilon_{\text{JL}}}{|\cos \angle(z_i, Hz_i)|}. \quad (12)$$

Proof. For each i with $v_i = Hz_i \neq 0$, apply Lemma A.1 to the pair $(x_i, y_i) = (z_i, v_i)$. This requires JL norm preservation on the set $\{z_i/\|z_i\|, v_i/\|v_i\|, z_i/\|z_i\| \pm v_i/\|v_i\|\}_{i=1}^n$, whose cardinality is at most $4n$. Standard JL finite-set bounds imply that (7) holds for all elements of this set with probability at least $1 - \delta$ provided $m \gtrsim \varepsilon_{\text{JL}}^{-2} \log((4n)/\delta)$, which is equivalent to (10) up to constants. Under this event, Lemma A.1 yields $|\langle Rz_i, Rv_i \rangle - \langle z_i, v_i \rangle| \leq \varepsilon_{\text{JL}} \|z_i\| \|v_i\|$, which is exactly (11) (since $v_i = Hz_i$). The relative form (12) follows by dividing by $|c_i| = |\langle z_i, v_i \rangle|$ and using $|\langle z_i, v_i \rangle| = \|z_i\| \|v_i\| \cos \angle(z_i, v_i)$. If $v_i = 0$, then $c_i = \tilde{c}_i = 0$ and the bounds hold trivially. $\square$

Remarks.

- **Linearity of R .** The definition $\tilde{c}_i = \langle Rz_i, R(Hz_i) \rangle$ uses only that R is linear so that $R(Hz_i)$ is well-defined as applying the same JL map to the vector Hz_i . This is standard for JL embeddings.
- **Relationship between m and d .** The guarantee is *finite-set*: m scales like $\log n$ (not d) because we only need geometry preservation on the particular vectors used in the step. Note that Hz_i is formed in the ambient $\mathbb{R}^d$ first (conceptually, in analysis), and then R is applied to z_i and Hz_i as vectors in $\mathbb{R}^d$. No claim is made that one can apply H in the compressed space.

- **Small-curvature regime.** The relative bound depends on $|\cos \angle(z_i, Hz_i)|$ and becomes loose when z_i lies near the null space of H (or, more generally, when z_i and Hz_i are nearly orthogonal). This is exactly why curvature-based weights must regularize near $c_i \approx 0$ (e.g., via a floor λ_{reg} and saturation exponent α).

Lemma A.3 (Bias of the 3-point curvature estimator (correct Lagrange remainder)). Assume L is three-times continuously differentiable on a neighborhood of the segment $\{\theta + tz : |t| \leq \epsilon_{\text{fd}}\}$. Define the 3-point estimator

$$\hat{c}(z) := \frac{L(\theta + \epsilon_{\text{fd}}z) - 2L(\theta) + L(\theta - \epsilon_{\text{fd}}z)}{\epsilon_{\text{fd}}^2}.$$

Let $H = \nabla^2 L(\theta)$. Then there exist $\xi_+ \in (0, 1)$ and $\xi_- \in (-1, 0)$ such that

$$\hat{c}(z) = z^\top Hz + \frac{\epsilon_{\text{fd}}}{6} \left(\nabla^3 L(\theta + \xi_+ \epsilon_{\text{fd}}z)[z, z, z] - \nabla^3 L(\theta + \xi_- \epsilon_{\text{fd}}z)[z, z, z] \right). \quad (13)$$

Consequently,

$$|\hat{c}(z) - z^\top Hz| \leq \frac{\epsilon_{\text{fd}}}{3} \sup_{|t| \leq \epsilon_{\text{fd}}} |\nabla^3 L(\theta + tz)[z, z, z]|. \quad (14)$$

In particular, if $|\nabla^3 L(u)[z, z, z]| \leq M\|z\|^3$ along the segment, then

$$|\hat{c}(z) - c(z)| \leq \frac{\epsilon_{\text{fd}}}{3} M\|z\|^3. \quad (15)$$

Proof. Let $f(t) = L(\theta + tz)$. Taylor's theorem about $t = 0$ to second order with Lagrange remainder gives

$$f(\pm \epsilon_{\text{fd}}) = f(0) \pm f'(0)\epsilon_{\text{fd}} + \frac{1}{2}f''(0)\epsilon_{\text{fd}}^2 \pm \frac{1}{6}f^{(3)}(\eta_{\pm})\epsilon_{\text{fd}}^3$$

for some $\eta_+ \in (0, \epsilon_{\text{fd}})$ and $\eta_- \in (-\epsilon_{\text{fd}}, 0)$. Plugging into the symmetric stencil cancels the odd first-order term and yields

$$\hat{c}(z) = f''(0) + \frac{\epsilon_{\text{fd}}}{6} (f^{(3)}(\eta_+) - f^{(3)}(\eta_-)).$$

Since $f''(0) = z^\top \nabla^2 L(\theta) z$ and $f^{(3)}(\eta) = \nabla^3 L(\theta + \eta z)[z, z, z]$, we obtain (13) by setting $\eta_{\pm} = \xi_{\pm} \epsilon_{\text{fd}}$. The bounds (14)–(15) follow immediately by taking absolute values. $\square$

Notation note. To avoid overloading symbols, this appendix uses ε_{JL} for the JL distortion level and ϵ_{fd} for the finite-difference probe radius (the SPSA step used in $\hat{c}(\cdot)$).

Combined guarantee (one-line takeaway). For the finite set $\{z_i\}_{i=1}^n$ used in a step, with probability at least $1 - \delta$ over the JL map R , simultaneously for all i ,

$$\begin{aligned} |\langle Rz_i, R(Hz_i) \rangle - \hat{c}(z_i)| &\leq \underbrace{\varepsilon_{\text{JL}} \|z_i\| \|Hz_i\|}_{\text{JL inner-product distortion}} \\ &+ \underbrace{\frac{\epsilon_{\text{fd}}}{3} \sup_{|t| \leq \epsilon_{\text{fd}}} |\nabla^3 L(\theta + tz_i)[z_i, z_i, z_i]|}_{\text{finite-difference bias}}. \end{aligned} \quad (16)$$

A.6. E. α sweep and failure-rate plots

Alpha	Iter	Best Test	Note
0.01	51	93.6%	× DIVERGED
0.1	50	94.5%	Stable
0.25	51	91.7%	Stable
0.5	50	89.7%	Stable
0.75	50	89.0%	Stable
1.0	52	89.2%	Stable

Table 5. Alpha ablation: stability and performance. Lower alpha (0.01) risks collapse; moderate alpha (0.1) creates stability without sacrificing peak accuracy.

A.7. E. ϵ sweep and failure-rate plots

Epsilon	Best Test	Status
1e-2	80.5%	Never improved
5e-3	86.5%	Improved slightly then diverged
1e-3	93.0%	Good
5e-4	93.6%	Very good
1e-4	94.5%	BEST (eps=lr)
5e-5	92.7%	Good
1e-5	80.5%	Never improved
5e-6	80.5%	Never improved

Table 6. Epsilon Ablation Results ($\text{lr} = 10^{-4}$ for all runs). Best performance at $\epsilon = 10^{-4}$. The range $10^{-5} > \epsilon > 10^{-3}$ converge well, outside this range they fail. Intuitively, this makes sense as we are measuring where we are stepping to vs. measuring farther or earlier than we are stepping to.

A.8. F. Systems appendix: Bit packing + Triton kernels

A.9. Wall-Clock Per Probe Generation and Step Comparisons on OPT-13B and 96 perturbations per step

Method	Probe gen (ms/pert)	96 perts (s)	Step (s)	Speedup (probe gen)	Speedup (step)
triton.bitpacked	102.1	9.8	21.7	4.9×	2.76×
pytorch.flat	246.6	23.7	35.6	2.0×	1.68×
pytorch	499.5	48.0	59.9	1.0×	1.00×

Table 7. Perturbation generation comparison. “Speedup (probe gen)” is computed from ms/pert relative to naive pytorch implementation. “Speedup (total)” is computed from end-to-end total time relative to original, per parameter pytorch implementation. Analysis done in series on a single A100 to isolate the impact of just the triton kernel.

Reference Triton code

```
@triton.jit
def _unpack_and_apply(w_ptr, packed_ptr, n_elements, alpha, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)
    offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    w = tl.load(w_ptr + offsets, mask=mask)
    byte_idx = offsets // 8
    bit_idx = offsets % 8
    packed_byte = tl.load(packed_ptr + byte_idx, mask=mask)
    bit = (packed_byte >> bit_idx) & 1
    sign = tl.where(bit == 1, 1.0, -1.0)
    tl.store(w_ptr + offsets, w + alpha * sign, mask=mask)

@triton.jit
def _unpack_and_accumulate(grad_ptr, packed_ptr, n_elements, coeff, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)
    offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    byte_idx = offsets // 8
    bit_idx = offsets % 8
    packed_byte = tl.load(packed_ptr + byte_idx, mask=mask)
    bit = (packed_byte >> bit_idx) & 1
    sign = tl.where(bit == 1, 1.0, -1.0)
    grad = tl.load(grad_ptr + offsets, mask=mask)
    tl.store(grad_ptr + offsets, grad + coeff * sign, mask=mask)
```